COMP30024 Artificial Intelligence Project Part B Report

Cascade Game-Playing Agent

Peter Lu and Aaron Shi

Introduction

Our final agent for Cascade is an iterative-deepening alpha-beta minimax player. The agent separates the placement phase from the play phase. During placement, it uses a deterministic centre-seeking heuristic to choose stable opening positions. During the play phase, it performs adversarial search over legal MOVE, EAT and CASCADE actions, using a weighted evaluation function to estimate non-terminal positions from the perspective of the root player.

The main design goal was to build a tactically reliable agent that could search several turns ahead while remaining within the project's runtime and memory constraints. To support this, we implemented a compact mutable game-state representation, in-place move application and reversal, Zobrist hashing, a repetition-aware transposition table, and domain-specific move ordering.

State Representation and Agent Architecture

The submitted agent follows the required **Agent** interface. In **Agent.__init__**, the agent stores its player colour, an initially empty internal game state, repetition counts, a maximum search depth, a default move budget, and a debug flag. In **Agent.action**, the agent first generates all legal actions from its current internal state. If the game is still in the placement phase, it chooses a placement action using a deterministic heuristic. Otherwise, it computes a time budget, sets a deadline, and calls iterative-deepening alpha-beta search. If search fails to produce a move before the deadline, the agent returns the first legal action as a safe fallback. In **Agent.update**, the action confirmed by the referee is encoded into the agent's internal action format, applied to the internal state, and recorded in the repetition table during the play phase.

Instead of using a sparse dictionary representation in the runtime search path, the final agent uses a compact SearchState representation. Occupancy is stored using two bitboards, **red_bits** and **blue_bits**, while stack heights are stored in a length-64 list indexed by **idx = row * 8 + col**. Empty cells are represented by the absence of a bit in both occupancy bitboards and a height value of zero. The state also stores each player's total token count, the play-phase turn count, the side to move, and a Zobrist hash key.

Actions are encoded as integers internally. PLACE actions use the raw board cell index, while MOVE, EAT and CASCADE actions use separate offset ranges based on source index and direction. This avoids repeated construction of referee action objects during search. Conversion functions are used only at the boundary between the referee interface and the internal search engine.

Game simulation uses in-place **make_move** and **unmake_move** operations. Each applied move produces an UndoRecord, allowing the exact previous state to be restored after recursively searching a child node. This avoids copying the complete board for every branch, which is important because Cascade can produce wide action sets and complex CASCADE interactions. MOVE actions either relocate a stack or merge it with a friendly stack. EAT actions replace an adjacent enemy stack when the attacker has sufficient height. CASCADE actions remove the source stack, place single tokens along the chosen direction, recursively push occupied cells, and eliminate stacks that are pushed off the board.

Search Algorithm

The final agent uses iterative-deepening alpha-beta minimax. At the start of a play-phase decision, the agent searches depth 1, then depth 2, and so on until either the maximum depth cap is reached or the time deadline is approached. The default maximum depth is 8, but the completed depth is dynamic because the search stops according to the available time budget.

The alpha-beta procedure alternates between maximizing and minimizing nodes. Scores are always evaluated from the fixed perspective of the root player, rather than from the side currently to move. This makes the evaluation consistent across alternating turns. At maximizing nodes, the agent chooses the action with the highest score; at minimizing nodes, it assumes the opponent will choose the action with the lowest score. Alpha-beta pruning is then used to avoid exploring branches that cannot affect the final minimax decision.

Terminal states are handled explicitly. A state is terminal if either player has no tokens, the play-phase turn limit is reached, the same board state with the same side to move occurs for a third time, or the current player has no legal actions. Winning by eliminating the opponent is assigned a large positive score, while being eliminated is assigned a large negative score. At the turn limit, the player with more tokens is favoured. Draws are scored with a small material-sensitive bias: the agent is discouraged from accepting a draw when it is ahead, but encouraged to accept one when it is behind. This makes the terminal scoring better aligned with practical game strategy than treating every draw as neutral.

Iterative deepening also improves move ordering. The best move found at a shallower depth is used as the preferred move at the next depth, increasing the likelihood that strong branches are searched first and therefore improving alpha-beta pruning.

Evaluation Function and Strategy

The evaluation function estimates non-terminal board states using a weighted sum of material, tactical, mobility, positional and repetition features.

The most important feature is material balance, computed as the difference between the agent's total tokens and the opponent's total tokens. Since the objective of Cascade is to eliminate the opponent's tokens, material advantage is the most direct proxy for progress. This feature receives the largest weight.

The tactical capture feature measures the total height of enemy stacks that can currently be captured by the agent. It rewards immediate EAT opportunities, especially when they remove larger enemy stacks. This helps the agent identify tactical positions that may not yet show a large material advantage but contain strong threats.

The mobility feature rewards positions with more useful legal options. It counts access to empty movement targets, friendly merge targets, capturable enemy stacks, and stacks with height at least two that can CASCADE. This is important because low-mobility positions are easier for the opponent to constrain, while high-mobility positions preserve tactical flexibility.

The stack-count feature compares the number of friendly and enemy stacks. This complements material balance. A player with the same number of tokens distributed across more stacks may have more board influence, while a player with too few stacks may be vulnerable to being blocked or pushed.

The centre-control feature gives higher value to stacks closer to the centre of the board. The weight for a cell is based on the inverse of its Manhattan distance from the board centre. This encourages positions that have access to more directions and are less immediately vulnerable to edge elimination.

The edge-exposure feature penalises occupation near the outer ring of the board. This is domain-specific to Cascade, because CASCADE actions can push stacks off the board. Edge and near-edge stacks are therefore tactically vulnerable, even when they appear materially safe.

Finally, the evaluation includes a repetition bias based on the current repetition count. This works with the terminal draw scoring to discourage repeating positions when the agent is materially ahead and to tolerate repetition when the agent is behind. This is important because the game has a threefold repetition rule, and ignoring this rule can cause a search agent to misjudge repeated tactical cycles.

Optimisation and Time Management

Several optimisations were implemented to make deeper adversarial search feasible.

First, alpha-beta pruning reduces the number of nodes that need to be searched compared with plain minimax. Its effectiveness depends heavily on move ordering, so the agent orders actions before search. The ordering prioritises the previously best move from iterative deepening, then EAT actions ranked by captured stack height, then CASCADE actions ranked by a cascade-pressure heuristic, followed by the remaining actions. This ordering reflects the tactical structure of Cascade: captures immediately reduce enemy material, while cascades can create high-impact pushing and edge-elimination threats.

Second, the agent uses a per-search transposition table. Each entry stores information about previously evaluated states. The key combines the Zobrist hash, the play-phase turn count, and a repetition-history signature. Including the repetition signature is important because two visually identical boards may have different strategic values if one of them is closer to triggering a threefold repetition draw.

Third, the mutable **SearchState** avoids excessive allocation. Search applies and reverses moves using **make_move** and **unmake_move**, rather than copying an entire board for each successor. This is especially important for CASCADE moves, where pushing chains can affect multiple cells.

Fourth, the agent uses deadline-based time management. A default per-move budget is adjusted based on the stage of the game, with larger budgets early in the play phase and smaller budgets later. This allows the agent to spend more time on strategically important early decisions while avoiding excessive accumulated CPU usage across a full game.

The final implementation uses only the Python standard library and project-provided referee types. It does not use NumPy, AIMA code, external code, internet access, disk storage, multiprocessing, or additional third-party libraries.

Performance Evaluation

We evaluated the agent using both Gradescope public tests and local opponent benchmarks. On Gradescope, the submitted agent passed both public colour configurations: test agent as Red versus our submission as Blue, and our submission as Red versus the test agent as Blue. This confirmed that the agent could play legal complete games through the official referee interface.

For local testing, we ran 50 logged games against three internal baselines. The results were:

Test Setting	Games Played	Agent as Red	Agent as Blue	Win Rate	Average Runtime
Versus random baseline	24	12/12	12/12	100%	Tested against seeded random legal play; average full-game runtime 2.2s.
Versus greedy baseline	14	7/7	7/7	100%	Tested tactical strength against one-step evaluation; average full-game runtime 2.3s.
Versus depth-3 version	12	6/6	6/6	100%	Compared against the depth-3 variant; average full-game runtime 5.8s.
Total	50	25	25	100%	Average runtime across all games: 3.1s

These results show that the final agent was robust against seeded random play, a one-step greedy baseline, and an earlier shallower search version. The comparison against the depth-3 version was particularly useful because it tested whether the additional search depth and engineering optimisations improved actual play, rather than only increasing computation.

We also used engine-level checks during development. These tested legal-action generation, state update consistency, make/unmake reversibility, terminal-state semantics, terminal scoring precedence, and repetition-signature updates. These checks were important because a strong search algorithm is only useful if its internal simulator matches the referee's rules.

The local evaluation has limitations. The opponent set was internal and relatively small, and it does not prove performance against the hidden benchmark suite. However, passing the public tests and

winning all 50 logged local games provides evidence that the final agent is legal, stable, and substantially stronger than simple random or one-step tactical play.

Limitations and Future Work

The main limitation is that search remains exponential in the branching factor. Although alpha-beta pruning, move ordering, bitboards and make/unmake reduce the practical cost, the agent still cannot search the full game tree. Its decisions therefore depend on the quality of the evaluation function at the depth frontier.

The placement phase is also heuristic-only. The centre-seeking placement rule is simple and deterministic, but it does not perform adversarial search over possible opening setups. A stronger version could search the placement phase jointly with expected play-phase consequences.

The transposition table is rebuilt for each move rather than reused persistently across turns. This keeps memory usage controlled and avoids stale repetition contexts, but it may miss opportunities to reuse search information across consecutive decisions.

The implementation also does not use MCTS, reinforcement learning, or learned evaluation weights. A large amount of time was spent hand tuning the evaluation function, which could have been mitigated through the use of fine tuning algorithms such as SPSA or Textel's Tuning Method. These could be explored in future work, but they would require careful benchmarking and additional engineering. A more practical next improvement would be selective tactical extensions for forcing captures and high-pressure cascades, because these are directly relevant to Cascade's tactical structure.

References and Declaration of AI Use

Declaration of use of AI: ChatGPT, Codex, GitHub Copilot, and Claude Code were used to help interpret the project specification, inspect the project structure, and organise this report draft. Final implementation decisions, code, testing, and report content were reviewed and completed by the authors.